

构建与运行：把 AMSS-NCKU 跑起来

目 录

1 引言：优化第一步是跑通 baseline	2
2 代码目录结构	2
3 构建系统	2
3.1 compile.sh 与 CMake	2
4 CMake 关键配置	3
4.1 AMSS_OPT 优化选项	3
4.2 CUDA 架构设置	3
4.3 三语言混合编译	3
5 运行流程	4
5.1 run.sh 与 AMSS_NCKU_Program.py	4
6 AMSS_NCKU_Input.py 关键参数	4
7 TwoPuncture 缓存	5
8 输出文件	5
9 CPU/GPU 模式切换	5
10 调试技巧	6
11 正确性检查	6
12 本章你将学会	6
13 要点速查	7
14 小结	7

1 引言：优化第一步是跑通 baseline

优化的第一步不是改代码，而是先跑通 **baseline**（基准版本），拿到一个可复现的基准结果。只有先跑通，才能测量性能、定位瓶颈、验证优化效果。本章我们从头开始，学习如何构建 AMSS-NCKU 程序、配置运行参数、执行完整流程并检查结果正确性。

直觉 不妨把优化想象成赛车调试：你必须先把车开上赛道跑一圈，记录每个弯道的时间和速度，才能知道哪里需要改进。如果车都发动不起来，再多的调优理论都是空谈。

本章我们先认识代码目录结构，再用 `compile.sh` 构建三个可执行文件，然后通过 `run.sh` 运行完整流程，最后检查输出结果的正确性。

2 代码目录结构

实验代码位于 `src/lab4/` 目录下，包含构建脚本、运行脚本、CMake 配置和源代码：

```
src/lab4/
compile.sh      构建脚本
run.sh          运行脚本
check.sh        正确性检查
CMakeLists.txt  CMake 配置
AMSS_NCKU_Input.py  输入参数
AMSS_NCKU_Program.py 运行 driver
src/
*.C             C++ 源文件
*.h             C++ 头文件
*.f90           Fortran 源文件
*_gpu.cu        CUDA 源文件
```

各文件的作用如下：

1. `compile.sh`：构建脚本，调用 CMake 编译源代码，生成三个可执行文件。
2. `run.sh`：运行脚本，调用 Python driver 执行完整流程。
3. `check.sh`：正确性检查脚本，将运行结果与 `golden` 真值对比。
4. `CMakeLists.txt`：CMake 配置文件，定义编译选项、语言和目标。
5. `AMSS_NCKU_Input.py`：用户编辑的输入参数文件，控制物理和计算参数。
6. `AMSS_NCKU_Program.py`：driver 脚本，读取输入参数并编排完整运行流程。
7. `src/`：源代码目录，包含 C++（`.C/.h`）、Fortran（`.f90`）和 CUDA（`*_gpu.cu`）源文件。

理解了目录结构，接下来我们看如何构建这些代码。

3 构建系统

3.1 `compile.sh` 与 CMake

构建通过 `compile.sh` 脚本完成，它内部调用 **CMake**（Cross-Platform Make）来管理编译过程。CMake 根据配置文件 `CMakeLists.txt` 自动生成 `Makefile`，再调用对应的编译器编译 C++、Fortran 和 CUDA 三种语言的源代码。

`./compile.sh`

构建成功后，在 `build/` 目录下生成三个可执行文件：

可执行文件	作用	输入	输出
TwoPunctureABE	求解初始时刻两个黑洞度规, 生成初值数据	parfile	初值数据文件
ABE	CPU 版主演化程序, 按 BSSN 方程推进时空	parfile + 初值	演化结果
ABEGPU	GPU 版主演化程序, 功能同 ABE 但用 GPU 加速	parfile + 初值	演化结果

直觉 不妨把构建过程想象成组装一台机器：CMake 是“装配图纸”，告诉系统需要哪些零件（C++、Fortran、CUDA）、怎么连接它们；`compile.sh` 是“装配工”，按照图纸把零件组装成三台可以独立运行的机器。

构建成功后，我们得到了三个可执行文件，接下来学习如何运行它们。

4 CMake 关键配置

4.1 AMSS_OPT 优化选项

CMakeLists.txt 中定义了 `AMSS_OPT` 变量，控制 C++ 和 Fortran 代码的编译优化级别。默认情况下它可能使用较低的优化级别（如 `-O0` 或 `-O1`），适合调试但不适合性能测试。正式运行和性能评估时应设为较高级别（如 `-O2` 或 `-O3`）。

优化级别直接影响 CPU kernel 的性能。`-O0` 不做任何优化，`-O3` 启用包括循环展开、向量化等激进优化。本实验的优化策略之一就是调整 `AMSS_OPT`，但要注意不同优化级别可能暴露或掩盖编译器 bug。

4.2 CUDA 架构设置

CMakeLists.txt 中通过 `CMAKE_CUDA_ARCHITECTURES` 指定目标 GPU 的计算能力。实验代码默认设为 80，对应 A100 GPU (sm_80)，但实验环境的目标 GPU 是 V100 (sm_70)，必须修改为 70 才能正确编译和运行：

```
set(CMAKE_CUDA_ARCHITECTURES 70)
```

如果架构不匹配，编译时可能报错（找不到对应架构的 PTX），或者运行时 GPU kernel 无法启动。这是构建阶段最常见的错误之一。

V100 的计算能力是 7.0，在 CMake 中写成 70；A100 是 8.0，写成 80。代码默认为 A100 编写，实验环境却用 V100，所以必须手动修改。

4.3 三语言混合编译

AMSS-NCKU 涉及 C++、Fortran 和 CUDA 三种语言，CMake 需要同时启用三种编译器（`g++`、`gfortran`、`nvcc`）。三语言混合编译的一个关键问题是链接兼容性（Linking Compatibility）：C++ 调用 Fortran 函数时需要处理名称修饰（name mangling）和参数传递

方式的差异，通常在 C++ 端用 `extern "C"` 声明 Fortran 函数。CMakeLists.txt 已经处理好了这些细节，你只需确保构建环境安装了全部三种编译器。

回到我们的问题：正确配置 CMake 是程序正确运行的前提，特别是 CUDA 架构必须匹配目标 GPU。配置好后，接下来学习如何运行程序。

5 运行流程

5.1 run.sh 与 AMSS_NCKU_Program.py

运行通过 `run.sh` 脚本完成，它内部调用 `AMSS_NCKU_Program.py` 这个 driver 脚本编排完整流程：

`./run.sh`

driver 脚本依次执行以下步骤：

1. 读取 `AMSS_NCKU_Input.py` 中的输入参数。
2. 根据参数生成 `parfile`（参数文件），传递给后续可执行文件。
3. 运行 `TwoPunctureABE` 求解初值，生成初值数据文件。
4. 根据 `GPU_Calculation` 参数运行 `ABE`（CPU）或 `ABEGPU`（GPU）进行主演化。
5. 整理演化结果，输出数据文件到指定目录。

直觉 不妨把运行流程想象成一条流水线：Python driver 是“车间调度员”，按顺序把任务分给不同的工作站；`TwoPunctureABE` 是“备料站”，准备好初始数据；`ABE` 或 `ABEGPU` 是“主生产线”，把初值一步步演化成最终结果。调度员负责协调各站之间的衔接。

理解了运行流程，接下来我们看如何配置关键参数。

6 AMSS_NCKU_Input.py 关键参数

`AMSS_NCKU_Input.py` 是用户最常编辑的文件，包含物理参数和计算参数。以下是几个最关键的参数：

```
GPU_Calculation = "no"
MPI_processes = 4
OMP_threads = 8
File_directory = "GW250118"
Output_directory = "GW250118/AMSS_NCKU_output"
```

各参数的含义：

1. **GPU_Calculation**：计算模式开关。“no”表示用 CPU 版本 `ABE`，“yes”表示用 GPU 版本 `ABEGPU`。这是切换 CPU/GPU 模式的唯一入口。
2. **MPI_processes**：MPI 进程数，控制并行规模。GPU 模式下建议设为 1，因为一张 GPU 由一个进程管理最简单；CPU 模式下可以设为多进程。
3. **OMP_threads**：导出为环境变量 `OMP_NUM_THREADS`，控制 OpenMP 线程数。但 `baseline` 代码未启用 OpenMP，仅修改此参数不会带来加速，需要同时修改 `kernel` 代码才能真正利用多线程。
4. **File_directory**：输入文件目录，存放 `parfile` 和初值数据。
5. **Output_directory**：输出文件目录，存放演化结果和日志。

`OMP_threads` 是一个容易踩坑的参数：你以为改了它就能加速，但 *baseline* 的 *Fortran kernel* 里没有 *OpenMP* 指令，环境变量设了也没用。要利用 *OpenMP*，需要在 *kernel* 中添加 `!$omp parallel do` 等指令，这是后续优化的任务之一。

7 TwoPuncture 缓存

TwoPuncture 初值生成是流水线的第一步，通常需要几分钟。在调试阶段，你可能反复运行程序来检查后续阶段，每次都重算初值很浪费时间。`run.sh` 提供了 `--twop-cache` 选项来缓存初值数据：

```
./run.sh --twop-cache
```

首次运行时计算并保存初值，后续运行直接从缓存读取，跳过 TwoPuncture 阶段。这在反复调试演化参数时能节省大量时间。

注意：缓存仅用于调试，正式计时和性能评估时不要使用。因为优化目标是端到端的完整流程，使用缓存会跳过 *TwoPuncture* 阶段，导致测得的运行时间不完整。提交实验报告时必须关闭缓存，测量包含初值生成在内的完整时间。

8 输出文件

运行完成后，结果输出到 `Output_directory` 指定的目录（通常为 `GW250118/AMSS_NCKU_output/`）和 `binary_output/` 目录下。以下是关键输出文件：

文件	内容
<code>bssn_BH.dat</code>	黑洞位置随时间的演化轨迹
<code>bssn_ADMQs.dat</code>	ADM 守恒量, 用于检查能量动量守恒
<code>bssn_psi4.dat</code>	引力波信号 ψ_4 的时间序列
<code>bssn_constraint.dat</code>	约束 violation, 监控数值稳定性
<code>Error.log</code>	运行错误日志, 排查问题时首先查看

其中 `bssn_constraint.dat` 特别重要：约束 violation 应该保持在较小范围内，如果它指数增长，说明数值演化出了问题（通常是网格太粗或时间步太大）。`Error.log` 则记录运行过程中的警告和错误信息，是排查问题的第一手资料。

知道输出文件在哪、包含什么，才能判断运行是否成功。接下来看如何在不同的硬件平台上运行。

9 CPU/GPU 模式切换

AMSS-NCKU 支持 CPU 和 GPU 两种计算模式，通过 `GPU_Calculation` 参数切换。两种模式在不同硬件平台上运行：

模式	GPU_Calculation	可执行文件	平台	架构
CPU	"no"	ABE	Kunpeng 920B	ARM
GPU	"yes"	ABEGPU	V100 节点	x86

CPU 任务运行在**鲲鹏 920B** (Kunpeng 920B) 处理器上，这是一款 ARM 架构的国产 CPU。GPU 任务运行在 V100 节点上，宿主机是 x86 架构，搭载 NVIDIA V100 GPU (计算能力 7.0)。

切换平台时，不仅要修改 `GPU_Calculation` 参数，还要确保在正确的硬件上提交任务。CPU 任务提交到 `Kunpeng` 队列，GPU 任务提交到 `V100` 队列。同时，`CUDA` 架构必须匹配：CPU 模式不需要 `CUDA`，GPU 模式需要设为 `70` (V100)。

10 调试技巧

在调试阶段，完整演化可能需要几十分钟，反复运行非常耗时。一个实用的技巧是缩短 `Final_Evolution_Time` 参数，只演化少量时间步来快速验证流程是否跑通：

```
Final_Evolution_Time = 10.0
```

这样几分钟内就能完成一次运行，快速检查参数配置、输出格式和基本正确性。确认流程无误后，再恢复完整的演化时间进行正式运行。

例 取一次 baseline 运行的阶段时间分解来看：TwoPuncture 初值生成约 3 分钟，主演化循环约 25 分钟，输出整理约 2 分钟，总计约 30 分钟。演化阶段占了约 83%，如果能把演化时间缩短 20%，总时间就能减少约 5 分钟，约 17% 的整体提升。这说明优化精力应该集中在演化阶段，而不是初值生成或输出整理。缩短 `Final_Evolution_Time` 正是为了在调试时快速验证演化阶段，而不必等 25 分钟。

提交实验报告前，务必恢复完整的 `Final_Evolution_Time`，否则测量到的性能数据不具参考价值。调试用短时间，提交用完整时间。

11 正确性检查

优化的底线是不改变物理结果。每次优化后，都需要运行 `check.sh` 脚本验证结果是否正确：

```
./check.sh
```

`check.sh` 将你的运行结果与预先准备的 **golden 真值** (Golden Truth) 对比，检查关键物理量（如黑洞轨迹、引力波形、约束量）是否在允许误差范围内一致。如果检查不通过，说明优化改变了物理结果，需要回退修改。

正确性检查通过后，才能比较优化前后的运行时间。如果正确性不通过，再快的运行也是无效的。始终记住：先正确，再优化。

12 本章你将学会

1. 用 `compile.sh` 构建三个可执行文件 (TwoPunctureABE、ABE、ABEGPU)，理解 CMake 的三语言混合编译。
2. 用 `run.sh` 运行完整流程，理解 Python driver 编排的五个步骤。

3. 配置 AMSS_NCKU_Input.py 中的 MPI、OpenMP、GPU 参数，理解 OMP_threads 在 baseline 下无效的原因。
4. 使用 --twop-cache 缓存 TwoPuncture 初值，加速调试迭代。
5. 定位输出文件 (bssn_BH.dat、bssn_psi4.dat、bssn_constraint.dat 等) 并理解其含义。
6. 切换 CPU/GPU 模式，匹配对应的硬件平台 (Kunpeng 920B 与 V100 节点)。

13 要点速查

概念	要点
代码位置	src/lab4/, 含 compile.sh, run.sh, check.sh
构建命令	./compile.sh, 调用 CMake
三个可执行文件	TwoPunctureABE 初值, ABE CPU 演化, ABEGPU GPU 演化
AMSS_OPT	C++/Fortran 优化级别, 正式运行用 -O3
CUDA 架构	默认 80 (A100), 改为 70 (V100)
运行命令	./run.sh, 调用 AMSS_NCKU_Program.py
GPU_Calculation	"no" = ABE (CPU), "yes" = ABEGPU (GPU)
MPI_processes	MPI 进程数, GPU 模式建议设 1
OMP_threads	导出为 OMP_NUM_THREADS, baseline 未启用 OpenMP
TwoPuncture 缓存	./run.sh --twop-cache, 仅调试用
输出目录	GW250118/AMSS_NCKU_output/, binary_output/
输出文件	bssn_BH.dat, bssn_psi4.dat, bssn_constraint.dat, Error.log
调试参数	缩短 Final_Evolution_Time, 提交前恢复
正确性检查	./check.sh 对比 golden 真值
CPU 平台	Kunpeng 920B (ARM)
GPU 平台	V100 节点 (x86, sm_70)

14 小结

本章从“优化第一步是跑通 baseline”出发，介绍了 AMSS-NCKU 的代码目录结构，用 compile.sh 构建三个可执行文件，理解了 CMake 的关键配置 (AMSS_OPT 优化级别和 CMAKE_CUDA_ARCHITECTURES 架构匹配)。我们通过 run.sh 运行完整流程，学习了 AMSS_NCKU_Input.py 中的关键参数 (GPU_Calculation、MPI_processes、OMP_threads)，掌握了 TwoPuncture 缓存和缩短演化时间的调试技巧，最后用 check.sh 验证结果正确性。跑通 baseline 后，后续章节将进入性能分析和优化策略，逐步提升 AMSS-NCKU 的运行速度。

讲义基于 HPC101 2025 Lab4 实验内容编写